

PianoDSL: A Graphical Domain-Specific Language for Musical Visualization

Esteban Alejandro Villalba Delgadillo, Daniel Felipe Barrera Suárez

Department of Computer Engineering
Universidad Distrital Francisco José de Caldas, Bogotá, Colombia
{evillalbad, dbarreras}@udistrital.edu.co

Abstract—This report presents PianoDSL, a domain-specific language (DSL) designed for music composition and visualization. It interprets simple text-based musical instructions into piano roll graphics using a compiler written in Python. The methodology integrates lexical, syntactic, and semantic analyzers, and the output is a time-pitch diagram representing note duration and placement. The project demonstrates how compiler concepts can be applied in educational tools for computational music.

Index Terms—Domain-specific languages, computational music, sound visualization, DSL, Pygame, piano roll.

I. INTRODUCTION

Domain-specific languages (DSLs) have proven their educational value by enabling complex concepts to be expressed through simplified syntax. In computer science, this paradigm is often explored by creating micro-languages that generate graphical outputs from minimalist text commands.

Our project applies this idea to the musical domain by developing an interpreter that converts commands such as:

```
play C4 for 1.0 at 0.0  
play E4 for 0.5 at 1.0
```

into *piano roll* visual representations, a standard format in digital music production.

II. LITERATURE REVIEW

Languages such as Sonic Pi [1], FoxDot [2], and TidalCycles have shown that it is possible to manipulate music in real time through code. However, many of these require advanced technical knowledge. Our approach aims for pedagogical accessibility through simplified design.

Additionally, libraries like Pygame allow sound playback in a straightforward way, making it feasible to combine symbolic notation, sound reproduction, and graphical visualization in educational environments.

III. BACKGROUND

This project is grounded in theoretical principles studied in Workshop 1 of the course, specifically finite automata, regular expressions, and context-free grammars. These concepts inform the design of the language's lexical and syntactic structure. The interpreter's ability to model rules for notes, durations, and repetitions stems directly from grammar modeling and derivation trees explored in class.

IV. OBJECTIVES

- Design and implement a symbolic interpreter for musical phrases.
- Enable textual instructions to be converted into piano roll graphics.
- Apply core compiler concepts (lexing, parsing, semantics) to a creative domain.
- Provide a foundation for future sound synthesis and interaction.

V. SCOPE

- Supports single-line, monophonic musical notation.
- Input based on duration and starting time.
- No graphical score or tempo customization.
- Visualization only, no audio output in current version.

VI. ASSUMPTIONS

- Notes are formatted as `[A-G] [|b]?[0-8]`.
- Durations and start times are positive real numbers.
- Input is well-formed and follows DSL syntax.

VII. LIMITATIONS

- No concurrency or polyphony.
- No tempo, volume, or instrument changes.
- Visualization limited to a piano roll, not staff notation.

VIII. METHODOLOGY

The system is composed of three main components based on classical compiler architecture:

A. Lexical Analysis

The lexical analyzer is responsible for scanning the input string and dividing it into a list of meaningful symbols (tokens). We use Python's `re` module to define regular expressions for each token type:

- **KEYWORD**: `\b(playforlat)`
- **NOTE**: `[A-G] (#b)?[0-8]`
- **NUMBER**: `\d+ (\.\d+)?`
- **SKIP** and **NEWLINE** tokens are ignored.

This phase produces a flat list of tokens that can be passed to the parser.

B. Syntactic Analysis

The parser verifies that the token sequence matches the grammar rules of PianoDSL. It uses a recursive descent strategy to validate the following grammar:

Grammar Definition (BNF):

```
# GRAMMAR DEFINITION FOR PIANO DSL:

# <S>          -> <INSTRUCTION> <S> | <INSTRUCTION>
# <INSTRUCTION> -> "play" <NOTE> "for" <NUMBER> "at" <NUMBER>
# <NOTE>        -> <LETTER> [<ACCIDENTAL>] <OCTAVE>
# <LETTER>      -> "A" | "B" | "C" | "D" | "E" | "F" | "G"
# <ACCIDENTAL>  -> "#" | "b"
# <OCTAVE>      -> "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8"
# <NUMBER>      -> <DIGITS> [ "." <DIGITS> ]
# <DIGITS>      -> <DIGIT> <DIGITS> | <DIGIT>
# <DIGIT>       -> "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

Figure 1. Context-free grammar for PianoDSL

If the syntax does not match, meaningful error messages are generated to help the user correct their input.

C. Semantic Analysis

The semantic analyzer processes the validated tokens to extract meaningful musical data. Each instruction is transformed into a tuple (note, duration, time). This structured data is used by the visualizer to render the final piano roll. The semantic phase also handles validation of values (e.g., ensuring durations are positive).

D. Visualization

Once semantic data is extracted, the system uses matplotlib to create a piano roll. Each note is displayed as a horizontal bar positioned by its pitch and start time, with length proportional to its duration.

IX. SYSTEM ARCHITECTURE

The system follows a classic compiler pipeline adapted for musical instructions. Figure 2 illustrates the process:

PianoDSL - System Architecture

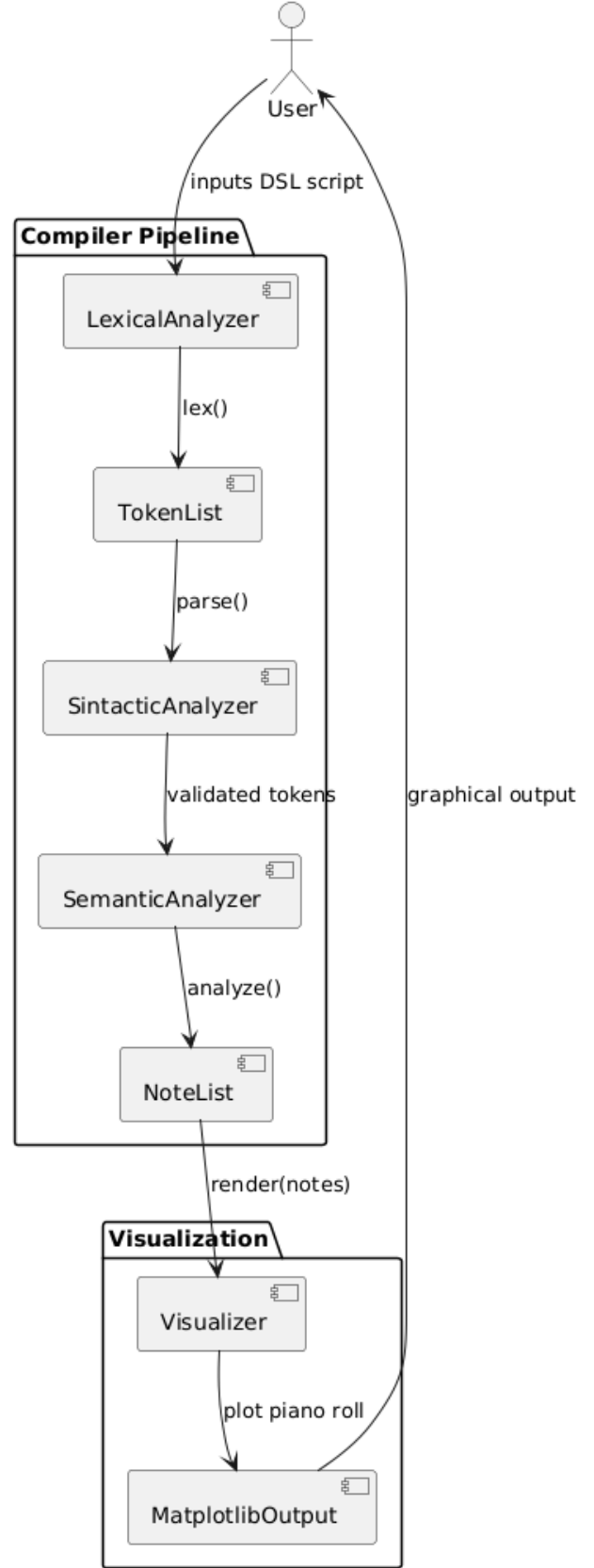


Figure 2. System architecture of the PianoDSL compiler

- **Input:** textual DSL code describing notes.
- **Lexical Analyzer:** splits input into tokens.
- **Syntactic Analyzer:** validates token structure using grammar rules.
- **Semantic Analyzer:** extracts musical data tuples.
- **Visualizer:** renders a piano roll from the musical data.

X. RESULTS

The following program was used as input:

```
play E4 for 1.0 at 0.0
play F4 for 1.0 at 1.0
play G4 for 1.0 at 2.0
play F4 for 1.0 at 3.0
play E4 for 1.0 at 4.0
```

The output was a piano roll diagram where each note appears horizontally with length proportional to its duration.

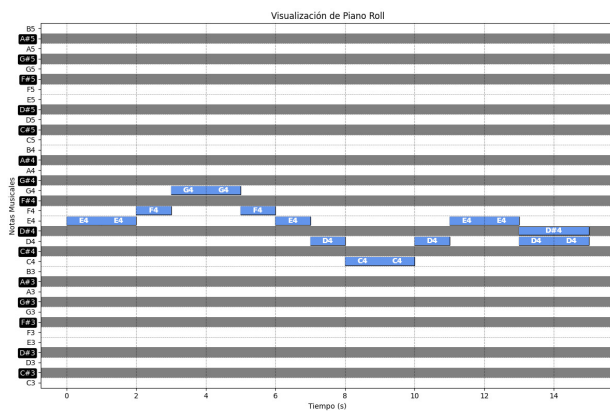


Figure 3. Sample piano roll visualization generated by the compiler

XI. TESTING AND VALIDATION

The compiler was tested using both valid and invalid input scripts.

A. Valid Input

Correctly structured scripts produced expected tokens, passed all grammar rules, and generated piano rolls without errors.

B. Invalid Input

We intentionally introduced errors (e.g., missing `for`, wrong note like `H9`) to verify error detection. The compiler returned meaningful exceptions that helped in debugging.

C. Token Verification

Sample input:

```
play D#4 for 1.5 at 6.0
```

Generated tokens:

- `KEYWORD(play)`
- `NOTE(D#4)`
- `KEYWORD(for)`

- `NUMBER(1.5)`
- `KEYWORD(at)`
- `NUMBER(6.0)`

XII. DISCUSSION

PianoDSL simplifies how users interact with music and programming. The consistent format promotes predictability, while the graphical output reinforces musical structure. Though basic, the interpreter demonstrates compiler theory applied in a creative and accessible context.

XIII. CONCLUSION

The project successfully integrates lexical analysis, parsing, and visualization into a unified tool. PianoDSL proves that a pedagogical compiler project can go beyond simple math or shapes and tap into musical creativity.

XIV. FOLDER STRUCTURE AND EXECUTION

The project is composed of the following Python modules:

- `lexical.py`: Handles token recognition.
- `syntactic.py`: Validates syntax using grammar rules.
- `semantic.py`: Extracts note data from tokens.
- `compiler.py`: Integrates all components and generates the visualization.

To execute the project:

```
python compiler.py
```

XV. FUTURE WORK

Planned improvements include:

- Integrating real-time sound playback using `pygame`.
- Supporting polyphony and chords.
- Adding loop instructions and tempo control.
- Exporting results to MIDI or MusicXML.
- Enhancing the interface with live editing capabilities.

ACKNOWLEDGEMENTS

We thank Eng. Carlos Sierra and the course team for their support, as well as the Universidad Distrital for encouraging interdisciplinary exploration.

REFERENCES

- [1] Sonic Pi. <https://sonic-pi.net>
- [2] FoxDot. <https://foxdot.org>
- [3] Pygame Documentation. <https://www.pygame.org/docs>